

ROOTSACADEMY 2026 · WEEK 1

Cloud, DevOps, Security & Docker 101

Carlos Bonalume · Dataroots · 3.5 hours

Welcome

Who is talking to you today

- **Carlos Bonalume** — role at Dataroots
- Clients / projects you worked on
- Your tech: cloud, IaC, languages
- Your role in the community (tech lead, competence lead, ...)
- One human fact, so they remember you

INPUT NEEDED

Quinten's version listed: name, role, clients (D'leteren, Telenet, KBC), tech (AWS / Terraform / platform automation / Go), and community role. Replace all five lines with yours, and add a photo if you want one.

Today

Four topics, one afternoon

01

Cloud

- What you rent, and from whom
- Service and deployment models
- ClickOps to IaC
- Architecture exercise

02

Containers

- Docker basics
- Layers and image size
- Orchestration
- Two hands-on exercises

03

DevOps

- Agile and DevOps
- Git branching
- CI/CD building blocks
- Pipeline exercise

04

Security

- Secrets
- Least privilege
- Zero trust
- Data responsibility

This is a high-level introduction. The goal is the vocabulary and the mental model, with small exercises to make it stick. You will go deeper in later modules and on your first project.

Two things you must not mix up

Principles

Why we work this way. These change slowly. Least privilege, automate everything, small reversible changes, own what you ship.

Tools

How we do it this year, at this client. Terraform, Docker, GitHub Actions, Kubernetes. These change fast.

Learn the principles. Look up the tools. Every client will hand you a different toolbox and expect you to recognise the shape of the problem.

Before we start

Quick warm-up: what do you already know?

- Five questions, anonymous, no wrong answers
- It tells you where you stand and tells me where to spend the time

INPUT NEEDED

Create your own Slido or Mentimeter event and put the join code here. The 2024 links in Notion point at Quinten's events and you cannot run those. Suggested questions: "Have you ever written a Dockerfile?", "Have you deployed to a cloud account?", "Do you know what a CI pipeline does?", "Have you used Terraform?", "Where do you keep secrets today?"

BLOCK 01 · 45 MINUTES

Cloud

What you actually rent, who is responsible for what, and the three ways to ask for it.

What is "the cloud"?

Answer in one sentence, without using the word "cloud".

What is cloud?

Computing resources you rent instead of own

- You rent compute, storage and networking from a provider, by the second or by the request
- Compare it with electricity at home. You do not own a power plant. You plug in, you use what you need, you get a bill
- Almost everything is sold as **something as a service**: IaaS, PaaS, SaaS, FaaS
- The provider operates the hardware. You operate what you put on it

What changes for you

- Capacity becomes a decision, not a purchase
- Mistakes cost money immediately
- Everything has an API, so everything can be automated
- Someone else's outage becomes your outage

Three properties that define it

If one is missing, it is not cloud

01

On demand

You get the resource in seconds, without a ticket, a phone call or a purchase order.

02

Pay as you go

You pay for what you use. You stop paying when you delete it. Forgetting to delete is the classic first bill shock.

03

API driven

Every action is an API call. The console, the CLI and Terraform all call the same API.

Property three is the one that makes DevOps possible. If infrastructure has an API, infrastructure can be code, and code can be reviewed, tested and rolled back.

Service models

How much of the stack do you still manage?

Model	You manage	Provider manages	AWS example
On premise	Everything, including the building	Nothing	your own server room
IaaS	OS, runtime, app, data	Hardware, virtualisation, network	EC2, EBS, VPC
PaaS	App, data, configuration	OS, runtime, scaling	Elastic Beanstalk, ECS Express, RDS
FaaS	Function code only	Everything else, including scale to zero	Lambda
SaaS	Your data and users	The whole product	Snowflake, Salesforce, Gmail

Moving down this table removes work and removes control. That trade is the whole architecture discussion in one line.

Deployment models

Whose environment are you running in?

Public cloud

You use resources from a provider in an environment shared with other customers. Isolation is logical, not physical. This is the normal case.

Private cloud

Your resources run in an environment isolated from other customers, on-premise or dedicated hardware. Common in banking and public sector.

Hybrid cloud

A mix of public and private. Usually the result of a migration that is not finished, or of data that legally cannot move.

Multi cloud

More than one provider at the same time. Sometimes strategy, more often the result of an acquisition. It doubles the operational surface.

Be careful with "multi cloud" as a goal. Portability sounds free and is not. You pay for it by using the lowest common denominator of every provider.

Shared responsibility model

The provider secures the cloud. You secure what you put in it

Provider: security OF the cloud

- Physical data centres and hardware
- Hypervisor and host OS
- Network backbone
- Managed service internals

You: security IN the cloud

- Identity, roles and permissions
- Network rules and public access
- Encryption choices and key handling
- Patching what you installed
- Your data and who can read it

Almost every cloud data breach you read about is on the right-hand side. A public storage bucket is not a provider failure.

The big three, and the others

Who you will actually work with

AWS

- The first large provider
- Largest worldwide
- Widest service catalogue
- Naming is chaos, docs are deep

Azure

- Microsoft
- Strongest position in Belgium
- Enterprise and hybrid focus
- Arrives with the Microsoft contract

GCP

- Google
- Strong data and ML services
- Smaller service catalogue
- Often a second provider, not the first

- Others you will meet: Oracle Cloud, IBM, Alibaba, and smaller platforms such as Snowflake and Databricks that run on top of the big three
- **European providers** (OVHcloud, Scaleway, Exoscale, StackIt) come up more and more in sovereignty and data-residency discussions

Same service, three names

Learn the concept once, then translate

Concept	AWS	Azure	GCP
Virtual machine	EC2	Virtual Machines	Compute Engine
Object storage	S3	Blob Storage	Cloud Storage
Serverless function	Lambda	Functions	Cloud Run functions
Container registry	ECR	Container Registry	Artifact Registry
Managed Kubernetes	EKS	AKS	GKE
Data warehouse	Redshift	Synapse / Fabric	BigQuery
Secrets store	Secrets Manager	Key Vault	Secret Manager
API gateway	API Gateway	API Management	API Gateway
Identity and permissions	IAM	Entra ID + RBAC	IAM
Managed CI/CD	CodePipeline	Azure DevOps	Cloud Build

You do not need to memorise this table. You need to know that it exists, so you never learn a service twice.

Three ways to talk to a cloud

ClickOps, CLI, infrastructure as code

1 · Console (ClickOps)

```
console.aws.amazon.com
```

- Best for looking around and learning
- Not repeatable
- Nobody can review a click

2 · CLI

```
$ aws sso login  
$ aws s3 mb s3://my-bucket  
$ aws lambda list-functions
```

- Scriptable, good for one-off tasks
- Still imperative
- No record of desired state

3 · Infrastructure as code

```
$ terraform plan  
$ terraform apply
```

- Declarative and reviewable
- Full lifecycle, including delete
- This is how we work

Rule of thumb: click to learn, script to explore, code to keep. Anything that must exist tomorrow belongs in code.

Infrastructure as code

Write code that creates cloud resources

```
resource "aws_s3_bucket" "landing" {  
  bucket = "ra2026-landing-${var.student}"  
}  
  
resource "aws_lambda_function" "counter" {  
  function_name = "file-counter"  
  package_type  = "Image"  
  image_uri     = var.image_uri  
  role          = aws_iam_role.counter.arn  
  timeout       = 30  
}
```

The file is the truth. The cloud is a projection of the file.

Why we do this

- **Reproducible** — dev, test and prod from the same code
- **Full lifecycle** — create, update and destroy
- **In git** — review, blame, rollback, and the reason for every change
- **Reviewable** — a colleague sees the change before the cloud does
- **Documenting** — the code is the only accurate diagram

Which IaC tool?

There are many flavours. The ideas are the same

Tool	Style	Use it when
Terraform / OpenTofu	Declarative HCL	Default choice. Works across providers, huge module ecosystem. What you will see most at clients
Pulumi , AWS CDK	Real programming language	The team is strong in Python or TypeScript and wants loops, types and tests
CloudFormation , ARM / Bicep	Provider native	Single-provider shop, or you need a feature the day it ships
Ansible	Configuration management	You must configure inside servers, not create them
Helm , Kustomize	Kubernetes manifests	The thing you deploy lives in a cluster

Terraform manages **state**: a file that maps your code to real resources. Where that state file lives, and who can lock it, is the first real decision on any project.

Demo

Terraform, from empty folder to running resource

terraform init → plan → apply → destroy

INPUT NEEDED

Pick your demo and prepare it. Suggestion that fits the later exercises: an S3 bucket plus a Lambda, about 20 lines. Show `plan` output carefully, change one line, plan again so they see the diff, then `destroy` so they see the delete side of the lifecycle. Have a recorded version ready. Live cloud demos fail on conference wifi.

Exercise · cloud architecture

20 min · pairs

Design it before you build it

REQUIREMENTS

- A REST API where a user can upload a file
- Uploaded files land on a storage location
- Every day, a job counts the files and writes the count to a database

How to work

- Draw it in draw.io or on paper, in pairs
- Name the AWS services you would use for each box
- Write down one thing your design does badly
- Be ready to defend one trade-off: cost, latency, or effort

There is no correct answer. There is a defended answer and an undefended one.

Architecture is about trade-offs

Cost, latency, operational effort, team skills, lock-in, and the deadline. You can improve any one of them. You cannot improve all of them.

BLOCK 02 · 60 MINUTES

Containerization

Docker, the layer model, and why your image is a gigabyte when it should be a hundred megabytes.

How we got here

Bare metal, virtual machines, containers, functions

1990S

Bare metal

One app, one physical server. Ordered months ahead. Used at 10 percent. "It works on that machine" and nowhere else.

2000S

Virtual machines

Many guest OSes on one host. Better use of hardware. Each VM still carries a full operating system, so it boots in minutes and weighs gigabytes.

2013+

Containers

Share the host kernel, isolate the process. Starts in under a second, weighs megabytes, and ships the dependencies with the app.

2014+

Functions

You give up the machine entirely and hand over a function. Scales to zero. You lose control over the runtime.

Each step trades control for speed and density. Containers are the current sweet spot for most data and AI workloads, which is why the rest of this block is about them.

What is a container?

An isolated process, not a small computer

Virtual machine

- Virtualises **hardware**
- Has its own guest kernel
- Boots in minutes, sized in gigabytes
- Strong isolation boundary
- Patch the OS inside it, forever

Container

- Virtualises the **operating system**
- Shares the host kernel
- Starts in under a second, sized in megabytes
- Weaker isolation: same kernel
- You replace it instead of patching it

- A container is one or more processes with their own view of the filesystem, network and process table
- Linux gives this with **namespaces** (what the process can see) and **cgroups** (how much it can use)
- On your Mac, Docker runs a small Linux VM. Containers are a Linux feature

What is Docker?

The tooling that made containers usable

- Open-source project that builds, runs and distributes containers in a **standard format**
- Containers existed before Docker. The standard format and the developer experience are what changed the industry
- It also runs the largest public registry, hub.docker.com
- The format is now an open standard (OCI), so other tools read and write the same images:
 - Podman, containerd, BuildKit, Buildah
 - Every cloud registry: ECR, ACR, Artifact Registry

The promise

"It works on my machine" becomes true everywhere, because the machine travels with the app.

The same image runs on your laptop, in CI, on ECS, and in Kubernetes. Nothing is rebuilt on the way.

Three words you must not mix up

Dockerfile, image, container

RECIPE

Dockerfile

A text file with build instructions. Lives in git next to your code. Reviewed like any other code.

ARTIFACT

Image

The built result: a read-only, immutable stack of layers plus metadata. You push this to a registry and tag it.

PROCESS

Container

A running instance of an image, with a thin writable layer on top. Disposable. Many containers from one image.

Dockerfile

— build →

image

— run →

container

A class is not an object. A Dockerfile is not an image, and an image is not a container. Interviewers ask this.

The three verbs

Build, push, run

```
# build a local image from the recipe, and tag it
$ docker build -t my-app:1.2.0 .

# log in to the registry and distribute the image
$ REGISTRY=$ACCOUNT.dkr.ecr.eu-west-1.amazonaws.com
$ aws ecr get-login-password --region eu-west-1 \
  | docker login --username AWS --password-stdin $REGISTRY
$ docker push $REGISTRY/my-app:1.2.0

# run it, map a port, pass configuration
$ docker run --rm -p 8000:8000 -e LOG_LEVEL=debug my-app:1.2.0

# the four commands you will use while debugging
$ docker ps                # what is running
$ docker logs -f <container> # what it is saying
$ docker exec -it <container> sh # get a shell inside it
$ docker image ls          # what you have built, and how big it is
```

Configuration comes in from the outside with `-e` or a file. An image that must be rebuilt to change an environment is a broken image.

Anatomy of a Dockerfile

Seven instructions cover almost everything

```
FROM python:3.12-slim      # the base you start from
WORKDIR /app               # cd, for every later step
COPY requirements.txt .    # bring files in
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
ENV PYTHONUNBUFFERED=1     # runtime config default
EXPOSE 8000                # documentation, not a firewall
USER app                   # stop running as root
ENTRYPOINT ["python", "-m", "src.main"]
```

Two things people get wrong

- **CMD** alone is a complete default command, overridden by `docker run <image> <args>`. **ENTRYPOINT** is a fixed prefix that always runs, and it turns **CMD** into default arguments. Convention: **ENTRYPOINT** for the binary, **CMD** for the flags
- Use the exec form `["python", "app.py"]`. The shell form receives **SIGTERM** and does not pass it on, so your container is killed hard on every deploy. Also handle **SIGTERM** in your code, or run with `--init`
- **EXPOSE** publishes nothing by itself. Only `-p` does, or `-P`, which publishes everything **EXPOSE** lists

Exercise 1 · appetizer

10 min · solo

Get your hands on the CLI

- Pull an official Python image from Docker Hub
- Run the container. What happens, and why does it stop straight away?
- Open an interactive shell inside the container and print the environment variables
- Find the image size with `docker image ls`. Write the number down, you will need it later

```
$ docker pull python:3.12
$ docker run python:3.12
$ docker run -it python:3.12 sh
# env
```

Exercise 2 · write a Dockerfile

20 min · pairs

From source code to a running image

- Clone the exercise repository
- Write a Dockerfile that:
 - uses the official Python image as a base
 - copies `main.py` and `requirements.txt` into the image
 - installs the required packages
 - runs `main.py` with Python when the image starts
- Build it, tag it `ra-docker:<your-name>`, and run it

INPUT NEEDED

Quinten's slide points at github.com/datarootsio/rootsacademy-2024-docker-101. Check that it still exists, then fork or copy it to a 2026 repository, so the students do not open pull requests against a two-year-old repo.

BLOCK 02B · 35 MINUTES · NEW THIS YEAR

Layers and image size

The single most common Docker task on a real project is making somebody else's image smaller and faster to build.

Your image is 1.4 GB and the build takes 9 minutes

The app is one Python file. Nobody wrote it that way on purpose. It grew, one line at a time, and nobody looked at the number.

Why size and build time matter

Four costs, all of them paid repeatedly

Speed

Every CI run pulls and pushes the image. Every scale-out pulls it again. A gigabyte on a cold node is minutes of waiting.

Cold starts

ECS and Kubernetes pull the whole image before your code runs, so size becomes user-facing latency. Lambda streams only the chunks it needs, so it suffers less, but the 10 GB cap is real.

Money

Registry storage, cross-region data transfer, and CI minutes. Small on one image, real across hundreds of builds a day.

Attack surface

Every package in the image is a package that can have a CVE. A build compiler in production is a tool for an attacker.

Small images are not about tidiness. They are faster, cheaper and safer, and the fix is usually 10 lines of Dockerfile.

An image is a stack of layers

Each instruction that changes the filesystem adds one layer

container writable layer	created by docker run
ENV / USER / ENTRYPOINT	0 B · metadata only
COPY src/ ./src/	180 KB
RUN pip install -r requirements.txt	~420 MB
COPY requirements.txt .	2 KB
WORKDIR /app	0 B · metadata only
FROM python:3.12-slim	~150 MB

What follows from this

- Layers are **read only** and stacked in order. The image is the sum of them
- `RUN`, `COPY` and `ADD` create layers. `ENV`, `WORKDIR`, `USER`, `LABEL` and `ENTRYPOINT` only write metadata
- Layers are content-addressed, so two images on the same base **share** those bytes on disk and in the registry
- `docker run` adds a thin writable layer. It disappears with the container. That is why data must go to a volume or to S3

Layers are cached, and the cache is ordered

One changed line invalidates everything below it

- Before running an instruction, Docker checks whether it already has a layer for **that instruction with those inputs**. If yes, it reuses it in milliseconds
- On the first miss, the cache is gone for **every instruction after it**. Order is everything
- For `COPY`, the input is the content of the copied files. Change one byte in one file and that layer, and all later layers, are rebuilt

Cache hit

```
=> [2/5] WORKDIR /app          CACHED
=> [3/5] COPY requirements.txt  CACHED
=> [4/5] RUN pip install ...    CACHED
=> [5/5] COPY src/ ./src/      1.1s
Total: 3s
```

Cache miss on step 3

```
=> [2/5] WORKDIR /app          CACHED
=> [3/5] COPY . .              2.3s
=> [4/5] RUN pip install ...    184.0s
=> [5/5] COPY src/ ./src/      1.1s
Total: 3m 8s
```

Order your Dockerfile by change frequency

Rarely changing things first, your source code last

SLOW: REBUILDS ON EVERY COMMIT

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
ENTRYPOINT ["python", "-m", "src.main"]
```

Editing one line of Python changes `COPY . .`, so `pip install` runs again. Three minutes, every commit, for nothing.

FAST: DEPENDENCIES CACHED

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
ENTRYPOINT ["python", "-m", "src.main"]
```

Dependencies only reinstall when `requirements.txt` changes. Normal commits rebuild the last layer only. Three seconds.

Same image, same result, same lines of code. Only the order changed. This is the highest-value Dockerfile habit you can build.

The deletion trap

Deleting a file in a later layer does not make the image smaller

STILL 1.2 GB

```
RUN apt-get update
RUN apt-get install -y build-essential
RUN pip install -r requirements.txt
RUN apt-get remove -y build-essential
RUN rm -rf /var/lib/apt/lists/*
```

ACTUALLY SMALLER

```
RUN apt-get update \
  && apt-get install -y --no-install-recommends build-essential \
  && pip install --no-cache-dir -r requirements.txt \
  && apt-get purge -y build-essential \
  && apt-get autoremove -y \
  && rm -rf /var/lib/apt/lists/*
```

- Layers only **add**. A delete in a later layer writes a marker that hides the file. The bytes stay in the earlier layer, and they still travel with the image
- To remove something, remove it in **the same** `RUN` that created it

This applies to secrets too. A token you copied in and deleted three lines later is still in the earlier layer: anyone who can pull the image extracts it with `docker save`. And `docker history` exposes anything you passed as `ARG` or `ENV`.

Measure before you optimise

Find the fat layer instead of guessing

```
$ docker image ls my-app
REPOSITORY    TAG       IMAGE ID       SIZE
my-app        1.2.0     8a1f3c9d2e77  1.41GB

# which layer is the expensive one?
$ docker history my-app:1.2.0 --human --format "table {{.Size}}\t{{.CreatedBy}}"
SIZE          CREATED BY
612MB         RUN pip install -r requirements.txt
438MB         RUN apt-get install -y build-essential gcc g++
121MB         COPY . .      <- the .git folder and the venv
9.13kB        COPY src/ ./src/
0B            ENV PYTHONUNBUFFERED=1

# byte-level view, and what is wasted
$ dive my-app:1.2.0
```

Rule: never optimise a Dockerfile you have not run `docker history` on. It lists layers in build order, not by size — pipe it through `sort -h`, or use `dive`.

Where the bytes come from

The usual suspects, in the order you will find them

Cause	Typical cost	Fix
Full base image instead of a slim one	+850 MB	Use <code>-slim</code> , or a distroless runtime
Build toolchain left in the final image	+400 MB	Multi-stage build
Package manager caches	+50 to 300 MB	<code>pip --no-cache-dir</code> , <code>rm -rf /var/lib/apt/lists/*</code> in the same <code>RUN</code>
<code>COPY . .</code> pulling in <code>.git</code> , <code>.venv</code> , data, notebooks	+100 MB to GBs	A real <code>.dockerignore</code> , and copy only what you need
GPU or ML wheels you do not use at runtime	+2 GB and up	Install the CPU wheel explicitly, split the requirements files

`torch` is the classic: the default wheel pulls in CUDA, several gigabytes of it. Infer on CPU? Install from the CPU index and the image drops by an order of magnitude.

Choose your base image on purpose

The first line of the Dockerfile decides most of the size

python:3.12		~1.0 GB
python:3.12-slim		~150 MB
python:3.12-alpine		~55 MB
distroless/python3		~55 MB

Uncompressed, order of magnitude, before your own dependencies. Check the current numbers yourself.

Start with `-slim`

Debian, glibc, no documentation or extra tooling. Python wheels install normally. This is the sensible default for data and AI work.

Alpine: read the small print

Alpine uses musl, not glibc. The big scientific packages now ship musl wheels, but plenty of others do not, and PyTorch ships none at all, so `pip` compiles from source. Builds get slow, images can end up **bigger**, and musl's allocator can cost you numeric performance.

Distroless and `scratch` ship no shell and no package manager (the `:debug` tags do). Excellent for security and size, painful to debug, and distroless gives you Debian's system Python, not the version you pick. Use them when the service is stable.

Multi-stage builds

Build in a fat image, ship a thin one

```
# ----- stage 1: build -----
FROM python:3.12 AS builder
RUN apt-get update && apt-get install -y --no-install-recommends gcc
RUN python -m venv /opt/venv
ENV PATH=/opt/venv/bin:$PATH
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ----- stage 2: runtime -----
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
ENV PATH=/opt/venv/bin:$PATH
COPY src/ ./src/
USER 1000
ENTRYPOINT ["python", "-m", "src.main"]
```

What happens

- The compiler, the headers and the build caches live in stage 1 only
- Only the named paths are copied forward. Everything else is thrown away
- The final image holds the runtime, the installed packages and your code. Nothing else
- Typical result on a real project: **1.4 GB down to 220 MB**, and the same app

Copy a **whole virtual environment**, not loose packages. It keeps the console scripts and survives a Python version bump. The same shape works for compiled languages and for `npm run build`.

.dockerignore

Two minutes of work, often the biggest single win

```
.git
.github
.venv
venv/
__pycache__/
*.pyc
.pytest_cache
.mypy_cache
node_modules
data/
notebooks/
*.ipynb
.env
*.pem
README.md
Dockerfile
```

- Everything in the build context is sent to the builder before the build starts. A 900 MB context makes every build slow, even a cached one
- `COPY . .` without a `.dockerignore` is how `.git`, local virtual environments and sample data end up in production
- Excluding `.env` and key files stops the most common accidental secret leak
- Check it: `docker build` prints `transferring context: ...` on the first line

If that number is more than a few megabytes, stop and fix the ignore file first.

Four more levers

Worth knowing once you have the basics

BuildKit cache mounts

```
RUN --mount=type=cache,target=/root/.cache/pip \  
  pip install -r requirements.txt
```

Keeps the download cache between builds without putting it in the image. Drop `--no-cache-dir` here: the two cancel each other out.

Build secrets

```
RUN --mount=type=secret,id=tok \  
  pip install --index-url \  
    "https://$(cat /run/secrets/tok)@pypi.acme.io/simple" -r req.txt  
# docker build --secret id=tok,env=PYPI_TOKEN .
```

Mounted as a **file** at `/run/secrets/<id>` for that step only, and your command must read it. Never use `ARG` for a token.

Platform matters

```
docker build --platform linux/amd64 -t my-app .
```

Your Mac builds `arm64`; Lambda and most ECS tasks run `amd64`. This mismatch is the `exec format error` you meet in week one.

Pin and scan

```
FROM python:3.12-slim@sha256:0e1f...  
docker scout cves my-app:1.2.0
```

A digest makes the build reproducible. A scan tells you what you are shipping. Both belong in CI.

Checklist: slimming an image

In this order, every time

- **1 · Measure.** `docker image ls`, then `docker history`.
Write the number down
- **2 · Fix the context.** Write a `.dockerignore`. Check the transferred size
- **3 · Fix the base.** Move to `-slim` or distroless
- **4 · Reorder.** Dependencies before source code
- **5 · Split.** Add a builder stage, copy only the artifacts forward
- **6 · Clean in place.** No caches, and delete in the same `RUN`
- **7 · Measure again.** Compare with step 1, and put the number in the pull request
- **8 · Keep it honest.** Fail CI if the image grows past a threshold

Steps 2, 3 and 5 give you most of the win. Do those three and stop when the image is small enough. Perfect images are not the goal.

Exercise 3 · slim a bloated image

25 min · pairs

Take a real 1.4 GB image below 250 MB

- Build the provided Dockerfile and record the size and the build time
- Run `docker history` and name the three biggest layers
- Get the image under **250 MB** without changing the application code
- Edit one line of `main.py` and rebuild. Get the rebuild under **10 seconds**
- Write two sentences on what you changed and what it bought you

INPUT NEEDED

This exercise is new, so the repository does not exist yet. It needs a deliberately bad Dockerfile: full `python:3.12` base, `COPY . .` as the first step, `apt-get install build-essential`, no `.dockerignore`, a committed `.venv` and a `data/` folder, plus `torch` with CUDA in `requirements.txt`. Add a solutions branch.

BLOCK 03 · 25 MINUTES

Container orchestration

One container on your laptop is a demo. Production needs something to keep many of them alive.

Is Docker enough to run a reliable, scalable application?

- How do you add scalability and redundancy to your containers?
- How do you handle updating your containers without downtime?
- How do you make the app inside a container reachable?
- How do you manage many apps and many containers together?

What an orchestrator does for you

The four questions, answered

Question	What the orchestrator gives you
Scalability and redundancy	Run N copies across machines, restart what dies, add copies when load rises
Updating without downtime	Rolling updates, health checks, and an automatic rollback when the new version fails
Reachability	Service discovery and load balancing, so callers use a stable name instead of an IP address
Many apps together	Scheduling on shared machines, resource limits, isolation, and one place for configuration and secrets

You describe the **desired state**. The orchestrator keeps reality equal to it. This is the same idea as infrastructure as code, applied to running processes.

Kubernetes in one slide

The vocabulary you need to follow a conversation

Objects

- **Pod** — one or more containers that always run together. The smallest unit
- **Deployment** — "keep 3 pods of this image alive, and update them safely"
- **Service** — a stable name and load balancer in front of pods
- **Ingress** — how traffic from outside gets in
- **ConfigMap / Secret** — configuration and credentials, injected at runtime. A **Secret** is base64, not encrypted
- **Namespace** — a logical folder, not a security boundary on its own

How it works

- You submit YAML that describes the desired state
- A control loop compares desired with actual and fixes the difference, forever
- That loop is why a deleted pod comes back and a scaled deployment stays scaled

The honest warning: Kubernetes is powerful and expensive to own. It is the right answer for many services and many teams. It is rarely the right answer for one Python job that runs nightly.

You will not always need Kubernetes

Pick the smallest runtime that fits the workload

AWS option	Good for	Watch out for
Lambda	Events, short jobs, spiky traffic, glue code	15-minute limit, cold starts, and the image needs the Lambda runtime client
ECS on Fargate	Long-running containers, no cluster to manage	Less flexible than Kubernetes, AWS-specific
ECS Express Mode	One web service from one container, fastest path	Limited control over networking. App Runner is closed to new customers, use this instead
EKS	Many services, many teams, portability, complex networking	You now own a platform. Budget for that
Batch	Heavy scheduled or parallel compute jobs	Not for request-response workloads

ECS, EKS and Batch run the same OCI image unchanged, so the choice is a runtime decision, not a rebuild. Lambda is the exception: its image needs the Lambda runtime client baked in.

Cloud-native applications

Built for the platform, not moved onto it

Lift and shift

- One large application on a few big VMs
- State on local disk
- Scaling means a bigger machine
- Releases are events, with a runbook and a weekend
- Configuration lives in files on the server

Cloud native

- Small services in containers, managed by a platform
- Stateless processes, state in managed services
- Scaling means more copies, automatically
- Releases are routine and reversible
- Configuration comes from the environment
- Logs, metrics and traces are built in, not added later

Migrating to the cloud without changing the design gives you the old problems with a new bill. The value comes from the second column.

BLOCK 04 · 45 MINUTES

Agile, DevOps & CI/CD

How a change gets from your laptop to production, and the culture that makes that safe.

What is Agile?

Short feedback loops instead of long plans

Waterfall

Analyse, design, build, test, release. Each phase finishes before the next starts. You learn whether you built the right thing at the end, when changing it is most expensive.

Agile

Build a small slice, show it, learn, adjust. Repeat every one or two weeks. You learn continuously, and the plan is allowed to change.

- The value is the **feedback loop**, not the ceremonies. Standups and sprints are the tools
- Agile makes a promise it cannot keep alone: it can only deliver in two weeks if releasing takes minutes, not months

That gap between "the team is ready" and "the change is live" is exactly the gap DevOps closes.

What is DevOps?

"A set of practices intended to reduce the time between committing a change to a system and the change being placed into normal production, while ensuring high quality."

Bass, Weber & Zhu — *DevOps: A Software Architect's Perspective*

Read it again and notice what is not in it: no tool, no job title, no cloud provider. Two things only — **speed** and **quality**, and the claim that you can have both.

DevOps is not about the tools

Three things it is not, and one thing it is

Not a tool

Buying Kubernetes and GitHub Actions does not make you a DevOps team. Tools support a way of working. They do not create it.

Not a role

A "DevOps engineer" who takes the deployment work away from developers has rebuilt the wall with a new job title.

Not a team

A separate DevOps department between Dev and Ops is the problem DevOps was invented to remove.

It is end-to-end responsibility

The team that builds it, ships it, monitors it and gets paged for it. Feedback from production reaches the person who can fix the cause. Everything else in this block exists to make that responsibility bearable.

Does it actually work?

Four measures, used across the industry

Deployment frequency

How often you release. Elite teams: on demand, many times a day. Low performers: once a month or less.

Lead time for change

Commit to production. Elite: under an hour. Low: one to six months.

Change failure rate

Share of releases that cause a problem. Faster teams are not less careful. They fail less.

Failed deployment recovery time

How fast you recover, called "time to restore" in older reports. Elite: under an hour. Small changes are easy to reverse.

These are the **DORA metrics**, from the yearly State of DevOps research. The classic finding was that speed and stability rise together, because each change is small and reversible. The recent reports complicate it: AI-assisted teams ship faster but are less stable, so the trade-off returns unless the delivery practices are strong.

So what is a DevOps engineer?

An enabler of the way of working

- Builds and maintains CI/CD pipelines so other people can ship safely
- Sets up monitoring, logging and alerting, so problems are seen before users report them
- Makes testing and security checks part of the pipeline, not a separate gate
- Improves infrastructure reliability: scaling, self-healing, cost, and recovery
- Builds the paved road, then teaches people to use it

The honest note

Many companies lack the culture but want the label, so "DevOps engineer" is often used for what is really a cloud or infrastructure engineer.

Read the job description, not the title. Ask who gets paged at 3 a.m. The answer tells you whether the culture is real.

The same idea, other domains

MLOps, DataOps, GitOps, GenAIOps

MLOps

Ship and monitor **models** the same way. Extra problems: data and model versioning, retraining, drift.

MLflow, SageMaker, Vertex AI

DataOps

Ship and monitor **pipelines** and datasets. Extra problems: data quality tests, lineage, backfills.

dbt, Airflow, Dagster

GitOps

Git is the only source of desired state. An agent in the cluster syncs reality to the repository.

Argo CD, Flux

GenAIOps

Same discipline for prompts, agents and RAG. Extra problems: evaluation, cost per call, guardrails, non-determinism.

LangFuse, evaluation suites

Every one of these is DevOps with a harder artifact to version. Learn the base discipline once, then add the domain problems.

Git branching models

How the team agrees to work together

GitFlow

Long-lived `develop` and `main`, plus release and hotfix branches. Built for **stability** and for software with versioned releases that customers install.

- Clear separation between what is released and what is in progress
- Long-lived branches drift, so merges get painful
- Slow. It fights continuous delivery

Trunk-based development

One `main` branch. Short-lived feature branches, merged within a day or two behind feature flags. Built for **agility**.

- Small merges, few conflicts, fast feedback
- Needs a strong CI pipeline and real test coverage
- What most modern teams and most Dataroots projects use

- Whatever the model: protect `main`, require a pull request, and require the pipeline to pass before merge
- Every team does it slightly differently. Keep it as simple as the team allows

CI: continuous integration

Keep the team's code mergeable and healthy

WHAT IT ANSWERS

- How does the team collaborate without blocking each other?
- How do we keep code quality high without a manual reviewer for everything?

HOW

- Merge small changes often, through pull requests
- On every push, run automatically:
 - linting and formatting: `ruff`, `terraform fmt`
 - type checks: `mypy`
 - tests and coverage: `pytest`
 - security and dependency scans
 - build the artifact, so you know it still builds
- Repository policies: protected `main`, required checks, required review

The point of CI is not the tests. It is that a broken change is found in minutes by a machine, not in a week by a colleague.

CD: two different words

Continuous delivery and continuous deployment

Continuous delivery

Every change that passes CI is **ready** to release, automatically deployed to a staging environment. Production still needs a human to press the button.

Where most client projects sit, and a good place to be.

Continuous deployment

Every change that passes CI goes to **production**, with no human step.

Needs real confidence: good tests, monitoring, feature flags and a fast rollback. Earn it, do not start with it.

commit → build & test → deploy to dev → deploy to staging → production

The environments are the same code with different variables. If dev and production differ by hand-made changes, you do not have a pipeline. You have a habit.

Anatomy of a pipeline

Two systems, one config file, four words

- **Version control** stores the code and emits events
 - GitHub, GitLab, Bitbucket, Azure DevOps
- **Build system** watches for events and runs the work
 - GitHub Actions, GitLab CI, Jenkins, Azure Pipelines
- You define it in a config file, usually YAML, stored in the repository
- Every tool works roughly the same way. Which one you use depends on the client

```
on:                                # TRIGGER: which events
  push:
    branches: [main]
  pull_request:

jobs:                               # JOB: runs on one machine
  test:
    runs-on: ubuntu-latest
    steps:                           # STEP: one thing at a time
      - uses: actions/checkout@v5    # pre-made ACTION
      - uses: astral-sh/setup-uv@v8
      - run: uv sync --frozen         # your own command
      - run: uv run pytest -q
  build:
    needs: test                      # ordering between jobs
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5    # every job starts empty
      - run: docker build -t my-app:${{ github.sha }} .
```

A real pipeline, end to end

What the exercise later today will look like

1 · On PR

lint, type check, unit tests,
build the image without
pushing

2 · On merge to main

build the image once, tag it,
push it to ECR

3 · Deploy to dev

`terraform apply` with the
new image tag

4 · Smoke test

call the deployed endpoint
and check the answer

5 · On git tag

retag the same image,
deploy to production

Note what is not happening

- The image is built **once**. Later stages retag it, they do not rebuild
- Nobody logs into a server
- Nobody runs `terraform apply` from a laptop

Also worth having

- Manual approval before production
- Continuous testing at every stage, not only at the start
- Release during business hours, when people are awake to fix it

Hard-won rules

Things every project learns the painful way

- **Never put secrets in code.** Have the pipeline fetch them from a secret manager, ideally by reference at runtime rather than as environment variables, which leak into logs and `docker inspect`. Assume anything in git is public one day
- **Build the artifact once.** Retag and promote the same image between environments. Rebuilding per environment means production runs bytes you never tested
- **Pin your versions.** Actions, base images, dependencies. Prefer a digest over a tag. A floating `latest` is a build that breaks on a day you changed nothing
- **Use matrix builds** when it matters: several Python versions, several platforms
- **Add a smoke test** after every deploy. One call that proves the thing is alive catches most bad releases in seconds
- **Keep the pipeline fast.** A pipeline over 10 minutes gets worked around, and a pipeline people work around protects nothing
- **Make it reproducible.** Anyone should be able to rerun the pipeline on an old commit and get the same result

Demo

A pipeline running, passing, and failing

INPUT NEEDED

Prepare a repository you can push to live. The most valuable part is the **failure**: break a test on purpose, show the red check blocking the merge, fix it, show it go green. Then show the deploy job and the resource appearing in the AWS console. Have screenshots ready as a fallback.

CI/CD takeaways

If you remember five things

- It is about **releasing software**: consistent, reproducible, standardised
- Every change to production comes from a pipeline. No exceptions, no manual fixes
- Pipelines are triggered by git events, so you can go back in time and reverse a mistake
- It defines how your team collaborates, and how quickly you can respond to a problem
- Get it wrong and you lose time and money continuously. Get it right once and it works for you every day

How this is done will vary from client to client. Keep it as simple as the team can maintain.

Exercise 4 · build a CI/CD pipeline

45 min · pairs

Deploy a Python Lambda to AWS, properly

PART 1

Continuous integration

- Lint Python and Terraform, run `pytest`
- Runs on every push
- Merge to main only through a PR
- Block the PR when the pipeline fails
- Bonus: matrix over two Python versions

PART 2

Continuous delivery

- Build the image, push it to ECR
- Deploy to dev with Terraform
- Tag `<env>--<version>` from `pyproject.toml`
- Runs only on push to main
- Bonus: smoke test the dev deploy

PART 3 · BONUS

Continuous deployment

- Retag the dev image, do not rebuild
- Deploy to production
- Runs only when a git tag is created
- Bonus: smoke test production

INPUT NEEDED

Two repos exist: `rootsacademy-cicd-exercise` (AWS) and the `-azure` variant. You chose AWS: check the first one still works and copy it to a 2026 repo. Confirm with the organisers that per-student IAM users and ECR repos exist before the session.

BLOCK 05 · 30 MINUTES

Security 101

Four principles. If you hold on to these, you will avoid most of the mistakes juniors make in their first year.

Four principles

Everything else follows from these

01

Handle secrets with care

Credentials never live in code, chat or a ticket.

02

Least privileged access

Give exactly the permission needed, on exactly the resource needed.

03

Zero trust

Verify every request. Assume the network is already hostile.

04

Treat data responsibly

Client data is not yours. Handle it as if the owner is watching.

You are a consultant. You will hold credentials to systems that matter, and clients will trust you with data before they know you. Earn that.

Handle secrets with care

The single most common junior mistake

Never

- Commit credentials to git, even in a private repository, even "temporarily"
- Paste a secret into Slack, email or a ticket
- Hardcode a key in a notebook you will "clean up later"
- Put a token in a Docker `ARG` or an image layer
- Share an account. Every human has their own identity

Instead

- Use a secret manager: AWS Secrets Manager, SSM Parameter Store, Azure Key Vault, HashiCorp Vault
- Inject at runtime as environment variables, or fetch by reference
- Use short-lived credentials: SSO, OIDC federation from CI, assumed roles
- Use a one-time link tool to share a secret with a person, for example `password.link`
- Add `gitleaks` or a pre-commit secret scanner to the pipeline

If you do leak one: say so immediately and **rotate it**. Deleting the commit is not enough, git keeps history and the internet keeps copies. Nobody at Dataroots will be angry that you reported it fast.

Demo

Storing and reading a secret without it touching your code

```
$ aws secretsmanager create-secret --name ra2026/demo/db-password --secret-string '<value>'

# the application asks for it by name at runtime, and never stores it
$ aws secretsmanager get-secret-value --secret-id ra2026/demo/db-password --query SecretString --output text
```

INPUT NEEDED

Quinten demoed Azure Key Vault. Since you chose AWS, redo this with Secrets Manager or SSM Parameter Store. Worth showing: the IAM policy that allows one role to read one secret, and the CloudTrail entry that records who read it. The audit trail is the part that surprises people.

Least privileged access

Grant the minimum, on the smallest resource, for the shortest time

- Start from **nothing** and add what is needed.
Never start from admin and remove
- Permissions belong to **roles**, not people.
People and applications assume roles
- Scope it three ways: which **action**, on which **resource**, for how **long**
- Review it. Permissions only ever grow unless somebody looks

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject"],
  "Resource": "arn:aws:s3:::ra-landing/incoming/*"
}

# not this
{
  "Effect": "Allow",
  "Action": "s3:*",
  "Resource": "*"
}
```

These are single **statements**. A real policy wraps them in `{"Version": "2012-10-17", "Statement": [ ... ]}`. Concrete version: Alice may read and write the `customers` table and only read `orders`. The application may read one prefix of one bucket. Bob gets read-only. Nobody gets `*`.

Zero trust

There is no safe inside

The old model

A firewall around the network. Inside is trusted, outside is not. One phished laptop, and the attacker is inside with everyone else.

- **1 • Continuous verification.** Authenticate and authorise every request, from every user and every service, every time. Not once at the door
- **2 • Least privilege.** Even a verified caller only gets what it needs
- **3 • Assume breach.** Design so that one compromised identity is contained: segment networks, log everything, keep credentials short-lived

Practically, at a client: no shared VPN password as the only control, service-to-service calls authenticated with their own identities, MFA everywhere, and access that expires by default. Working with an external partner is a normal case, not an exception you build a hole for.

Treat data responsibly

Four rules to apply from day one

- **No public or anonymous access** unless it is genuinely needed and deliberately chosen. Check the default. Some services are open until you close them
- **No customer data on your laptop.** Not in a CSV in Downloads, not in a notebook, not "just a sample". Laptops get lost and get backed up to places you did not choose
- **Report GDPR concerns.** If you see personal data where it should not be, or data kept longer than needed, say so. You are not expected to solve it, only to raise it
- **Work with anonymised or synthetic data** whenever the task allows it. Most development and all demos do

If you are not sure: ask your manager or tech lead **before** you copy the file. One question costs a minute, and a data incident costs a client relationship.

WRAP UP

Putting it together

Principles on the left, tools on the right. You will meet different tools. The principles travel with you.

Principles and tools

One picture of the whole afternoon

Principle	What it means in practice	Tools you will meet
Rent, do not own	On demand, pay as you go, everything has an API	AWS, Azure, GCP
Describe, do not click	Desired state in git, applied by a machine	Terraform, Pulumi, CDK
Ship the environment	Build once, run anywhere, keep the image small	Docker, ECR, BuildKit
Declare the running state	The platform keeps N copies alive, not you	ECS, EKS, Kubernetes, Lambda
Small changes, often	Short-lived branches, automated checks, fast rollback	Git, GitHub Actions
Own it end to end	The team that builds it, runs it and is paged for it	monitoring, alerting, DORA
Least privilege, always	Minimum permission, short-lived, no secrets in code	IAM, Secrets Manager, SSO

Quick check

Same questions as this morning. Did anything change?

INPUT NEEDED

Create your own Mentimeter or Slido for the closing quiz and put the code here. The Notion link points at Quinten's 2024 event. Reusing the opening questions is the strongest ending: they see their own progress in one screen.

Where to go next

Free, short, and worth your evening

Docker

- Docker docs: build best practices
- `dive` — inspect any image layer by layer
- Try converting one of your own projects to multi-stage

Cloud & IaC

- AWS Well-Architected Framework, the five pillars
- Terraform tutorials on developer.hashicorp.com
- Build the architecture you drew today, then destroy it

DevOps

- State of DevOps / DORA report
- *The Phoenix Project* — a novel, and the fastest way to feel why this matters
- Look through the Dataroots GitHub for real pipelines

Ask in `#techno-ask` on Slack. Everybody there has broken a pipeline and leaked something once. Asking early is the behaviour we want, not a weakness.

CLOUD · DEVOPS · SECURITY · DOCKER 101

Questions?

Thanks for your attention — find me on Slack

Before you present: your to-do list

Every slide in this deck that needs your input

Slide	What	Action
2	Your intro	Replace Quinten's name, role, clients and tech with yours
5	Warm-up quiz	Create your own Slido or Mentimeter, add the join code
18	Terraform demo	Choose the demo, rehearse it, record a fallback
29	Docker exercise repo	Fork <code>rootsacademy-2024-docker-101</code> to a 2026 repo
44	Image slimming exercise	New. Create the deliberately bloated repo plus a solutions branch
64	CI/CD demo	Prepare a repo where you can break and fix a test live
66	CI/CD exercise	Copy the AWS exercise repo to 2026, confirm IAM users and ECR repos exist
70	Secrets demo	Rebuild Quinten's Key Vault demo on AWS Secrets Manager
76	Final quiz	Reuse the opening questions in your own event
all	Timing	Confirm your slot length with the organisers, then adjust the exercise time boxes

Slides needing your input are marked with a blue bar at the top and a dashed box. Press I to jump through them.